Exercício 11.4: Desafio: App-of-Apps e CI que atualiza a tag

Aluno: Luiz Brito · **Data:** 25/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- kind com Kubernetes 1.36.1, Argo CD v3.5.6, Gitea 1.27 no cluster (repositórios `fiap/fiap-store-config` e `fiap/store-api`), Sealed Secrets v0.40.0 e **Knative Serving v1.23.0 com Kourier**, instalado de verdade, então o componente serverless roda.
- **Sem GitHub Actions no lab** (opção "só local"). O workflow `.github/workflows/ci.yaml` está pronto para o GitHub. Os mesmos passos foram executados por `03-simula-ci.sh`, que troca só o `docker push` por `kind load` e a action `create-pull-request` pela API do Gitea. O PR foi aberto pelo script, não por um runner de CI. Essa é a ressalva honesta sobre o critério "PR criado pelo CI sem intervenção manual".

Parte 1: App-of-Apps

Por que App-of-Apps e não ApplicationSet: a frota tem 4 peças **heterogêneas**: infra (namespaces), dev e prod (mesmo código, overlays diferentes) e serverless (outro tipo de recurso, Knative). O ApplicationSet brilha quando N Applications saem do mesmo molde, e aqui o molde não se repete. O App-of-Apps também é o padrão em que `sync-wave` entre Applications é nativo: o root aplica as Applications como recursos comuns e respeita a ordem das waves. O custo é ter um YAML por Application em `apps/`, aceitável para 4 itens. Se a loja crescesse para dezenas de ambientes iguais, a escolha mudaria para ApplicationSet com gerador `git`.

Árvore do repositório de config

```
.
├── apps
│   ├── infra.yaml
│   ├── project.yaml
│   ├── store-dev.yaml
│   ├── store-prod.yaml
│   └── store-serverless.yaml
├── base
│   ├── deployment.yaml
│   ├── kustomization.yaml
│   ├── presync-valida.yaml
│   ├── redis.yaml
│   ├── service.yaml
│   ├── store-front.yaml
│   ├── store-relatorio.yaml
│   └── store-worker.yaml
├── infra
│   ├── kustomization.yaml
│   └── namespaces.yaml
└── overlays
    ├── dev
    │   ├── kustomization.yaml
    │   └── sealed-db-credentials.yaml
    ├── prod
    │   ├── kustomization.yaml
    │   ├── patch-prod.yaml
    │   ├── pdb.yaml
    │   └── sealed-db-credentials.yaml
    └── serverless
        ├── kustomization.yaml
        └── service.yaml
```

Ponto de entrada único

```yaml
# unico ponto de entrada: kubectl apply -f 02-root.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: fiap-store-root
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default # o root cria AppProject e Applications no namespace argocd, fora do fiap-store
  source:
    repoURL: http://gitea-http.gitea.svc.cluster.local:3000/fiap/fiap-store-config.git
    targetRevision: main
    path: apps
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

`apps/` contém o AppProject (wave -2), a `infra` (wave -1) e `store-dev`, `store-prod` e `store-serverless` (wave 0):

```yaml
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: fiap-store
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  description: FIAP Store, so o repo de config e so os namespaces da loja
  sourceRepos:
    - http://gitea-http.gitea.svc.cluster.local:3000/fiap/fiap-store-config.git
  destinations:
    - server: https://kubernetes.default.svc
      namespace: store-dev
    - server: https://kubernetes.default.svc
      namespace: store-prod
    - server: https://kubernetes.default.svc
      namespace: store-serverless
  clusterResourceWhitelist:
    - group: ""
      kind: Namespace
```

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: infra
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-1"
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: fiap-store
  source:
    repoURL: http://gitea-http.gitea.svc.cluster.local:3000/fiap/fiap-store-config.git
    targetRevision: main
    path: infra
  destination:
    server: https://kubernetes.default.svc
    namespace: store-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: store-dev
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "0"
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: fiap-store
  source:
    repoURL: http://gitea-http.gitea.svc.cluster.local:3000/fiap/fiap-store-config.git
    targetRevision: main
    path: overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: store-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

`store-prod` e `store-serverless` são iguais, mudando `path` e `namespace`. A `infra` cria os 3 namespaces com os labels de PSA do capítulo 10, as "policies" da plataforma. O Kyverno do enunciado foi substituído pelo PSA nativo, que é o que os exercícios do capítulo 10 usaram.

Sync-wave entre Applications exige health de Application. Desde o Argo CD 1.8, uma Application não tem health próprio, e o root não esperaria a wave -1 ficar saudável antes da wave 0. Foi restaurado o health check oficial (doc *Resource Health → Argocd App*) no `argocd-cm`:

```
# health de Application restaurado (doc oficial "Argocd App"): sem isso o root nao
# espera a wave -1 (infra) ficar Healthy antes de criar a wave 0 (apps)
data:
  resource.customizations.health.argoproj.io_Application: |
    hs = {}
    hs.status = "Progressing"
    hs.message = ""
    if obj.status ~= nil then
      if obj.status.health ~= nil then
        hs.status = obj.status.health.status
        if obj.status.health.message ~= nil then
          hs.message = obj.status.health.message
        end
      end
    end
    return hs
```

Hook PreSync

```yaml
# hook PreSync: roda antes de cada sync; se falhar, o sync para aqui
apiVersion: batch/v1
kind: Job
metadata:
  name: valida-conectividade
  annotations:
    argocd.argoproj.io/hook: PreSync
    argocd.argoproj.io/hook-delete-policy: HookSucceeded
spec:
  backoffLimit: 1
  template:
    spec:
      restartPolicy: Never
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        runAsUser: 65532
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: valida
          image: busybox:1.37
          # didatico: o "store-db" da loja ainda nao existe; valida DNS e a API do cluster
          command: ["sh", "-c", "echo validando conectividade antes do deploy; nslookup
kubernetes.default.svc.cluster.local && echo ok"]
          resources:
            requests:
              cpu: 10m
              memory: 16Mi
            limits:
              memory: 32Mi
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
```

Com `hook-delete-policy: HookSucceeded`, o Job some quando passa, e a reexecução no próximo sync não colide com o Job anterior.

Um único `kubectl apply`

O ambiente foi limpo antes (Applications antigas e namespaces `store-dev` / `store-prod` apagados):

```
21:42:14 $ kubectl apply -f 02-root.yaml
application.argoproj.io/fiap-store-root created

21:42:20 apps=5 synced+healthy=0
21:42:41 apps=5 synced+healthy=2
21:42:51 apps=5 synced+healthy=5
```

Ordem de criação, que comprova as sync-waves:

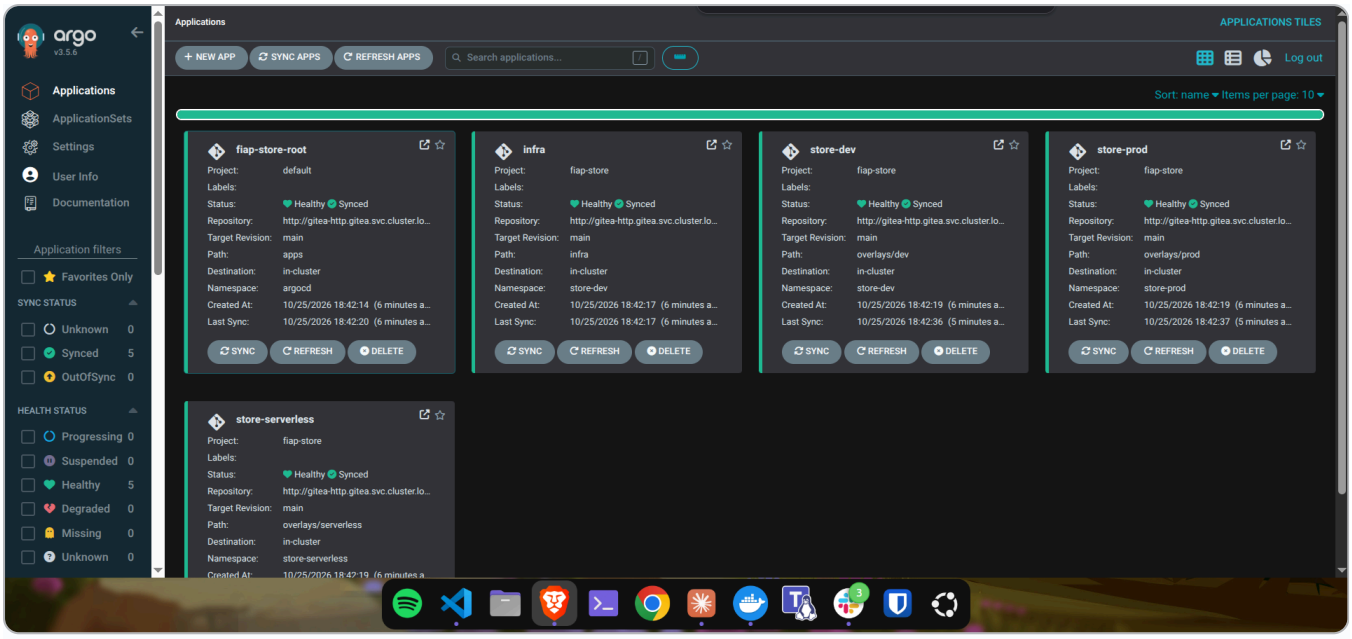
```
2026-10-25T21:42:14Z Application/fiap-store-root
2026-10-25T21:42:17Z Application/infra             wave=-1
2026-10-25T21:42:17Z Namespace/store-dev
2026-10-25T21:42:17Z Namespace/store-prod
2026-10-25T21:42:17Z Namespace/store-serverless
2026-10-25T21:42:19Z Application/store-dev         wave=0
2026-10-25T21:42:19Z Application/store-prod        wave=0
2026-10-25T21:42:19Z Application/store-serverless  wave=0
2026-10-25T21:42:36Z Deployment/store-dev/*        (depois do PreSync)
```

O AppProject `fiap-store` já existia desde o 11.2 e foi **adotado** pelo root, sem recriação.

PreSync executado antes dos Deployments:

```
$ kubectl -n argocd get application store-prod -o json | jq
'.status.operationState.syncResult.resources[] | select(.hookType)'
Job/valida-conectividade hookType=PreSync hookPhase=Succeeded msg=Reached expected number of
succeeded pods
$ kubectl get events -n store-prod --field-selector involvedObject.kind=Job
Normal   SuccessfulCreate   job/valida-conectividade   Created pod: valida-conectividade-h96sf
Normal   Completed          job/valida-conectividade   Job completed
$ kubectl get jobs -n store-prod
No resources found in store-prod namespace.      ← apagado pelo HookSucceeded
```

```
$ argocd app list
NAME                     NAMESPACE        PROJECT     STATUS  HEALTH   SYNCPOLICY
argocd/fiap-store-root   argocd           default     Synced  Healthy  Auto-Prune
argocd/infra             store-dev        fiap-store  Synced  Healthy  Auto-Prune
argocd/store-dev         store-dev        fiap-store  Synced  Healthy  Auto-Prune
argocd/store-prod        store-prod       fiap-store  Synced  Healthy  Auto-Prune
argocd/store-serverless  store-serverless fiap-store  Synced  Healthy  Auto-Prune
```



Configurações distintas por ambiente

```
$ kubectl get all -n store-dev
NAME                                    READY   STATUS    RESTARTS   AGE
pod/redis-d54bbc654-8jwh4               1/1     Running   0          9m55s
pod/store-api-849dc5655d-8fzlx          1/1     Running   0          32s
pod/store-front-cddfc5785-sjb95         1/1     Running   0          9m55s
pod/store-relatorio-685cf86b68-x4k46    1/1     Running   0          9m55s
pod/store-worker-6dcd699c76-trx6x       1/1     Running   0          9m55s

NAME                  TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
service/redis         ClusterIP   10.96.34.136    <none>        6379/TCP   9m55s
service/store-api     ClusterIP   10.96.193.137   <none>        80/TCP     9m55s
service/store-front   ClusterIP   10.96.81.48     <none>        80/TCP     9m55s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/redis               1/1     1            1           9m55s
deployment.apps/store-api           1/1     1            1           9m55s
deployment.apps/store-front         1/1     1            1           9m55s
deployment.apps/store-relatorio     1/1     1            1           9m55s
deployment.apps/store-worker        1/1     1            1           9m55s

NAME                                          DESIRED   CURRENT   READY   AGE

$ kubectl get all -n store-prod
NAME                                    READY   STATUS    RESTARTS   AGE
pod/redis-d54bbc654-qqx5n               1/1     Running   0          9m55s
pod/store-api-746df84ddc-4z44k          1/1     Running   0          9m55s
pod/store-api-746df84ddc-b8sfg          1/1     Running   0          9m55s
pod/store-api-746df84ddc-d9t2n          1/1     Running   0          9m55s
pod/store-api-746df84ddc-vfx2f          1/1     Running   0          9m55s
```

```
pod/store-front-cddfc5785-4dhn6        1/1     Running   0          9m55s
pod/store-front-cddfc5785-f9zbv        1/1     Running   0          9m55s
pod/store-front-cddfc5785-ntl9n        1/1     Running   0          9m55s
pod/store-relatorio-685cf86b68-6jc4s   1/1     Running   0          9m55s
pod/store-worker-6dcd699c76-49k6s      1/1     Running   0          9m55s
pod/store-worker-6dcd699c76-jgcbv      1/1     Running   0          9m55s

NAME                  TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
service/redis         ClusterIP   10.96.145.211   <none>        6379/TCP   9m55s
service/store-api     ClusterIP   10.96.54.53     <none>        80/TCP     9m55s
service/store-front   ClusterIP   10.96.159.7     <none>        80/TCP     9m55s

NAME                                 READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/redis                1/1     1            1           9m55s
deployment.apps/store-api            4/4     4            4           9m55s
deployment.apps/store-front          3/3     3            3           9m55s
deployment.apps/store-relatorio      1/1     1            1           9m55s
deployment.apps/store-worker         2/2     2            2           9m55s

NAME                                           DESIRED   CURRENT   READY   AGE
```

	`store-dev`	`store-prod`
store-api	1 réplica, tag `3` (bump do CI)	4 réplicas, tag `1.1`
store-front	1	3
store-worker	1	2
PSA	`baseline`	`restricted`
PDB, grace 60 s, `APP_ENV=prod`	não	sim

Parte 2: CI que promove por PR

```yaml
# CI da store-api: testa, constroi, escaneia, publica e PROPOE o deploy por PR no repo de config.
# Nao ha KUBECONFIG nem kubectl aqui: quem aplica no cluster e o Argo CD, lendo o Git.
name: ci

on:
  push:
    branches: [main]

env:
  IMAGEM: ghcr.io/${{ github.repository_owner }}/store-api
  TAG: ${{ github.run_number }}
```

```yaml
jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write # push no GHCR com o GITHUB_TOKEN
    steps:
      # actions fixadas por SHA do commit (tag e ponteiro movel; ver incidente do Trivy em 03/2026)
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1

      - uses: actions/setup-python@6d4fd22b20465ccc0f7354476588e8df7c9ca0e0 # v7.1.0
        with:
          python-version: "3.13"

      - name: Testes
        run: |
          pip install -r requirements.txt -r requirements-dev.txt
          pytest -q

      - name: Build
        run: docker build --build-arg APP_VERSION=$TAG -t $IMAGEM:$TAG .

      - name: Scan (porta de qualidade)
        # imagem oficial do Trivy com versao fixa, no lugar da trivy-action
        run: |
          docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
            aquasec/trivy:0.75.0 image --exit-code 1 --severity CRITICAL --ignore-unfixed
$IMAGEM:$TAG

      - uses: docker/login-action@81c6f93d4bc991e510b342a00b419b3bb4efc5c6 # v4.7.0
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Push
        run: docker push $IMAGEM:$TAG

  promove-dev:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - name: Clona o repo de config
        uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          repository: ${{ github.repository_owner }}/fiap-store-config
          token: ${{ secrets.CONFIG_REPO_TOKEN }} # PAT escopo repo: o GITHUB_TOKEN nao cruza
```

```
repositorios

      - name: Instala kustomize
        run: |
          curl -sLo k.tgz https://github.com/kubernetes-
sigs/kustomize/releases/download/kustomize%2Fv5.8.3/kustomize_v5.8.3_linux_amd64.tar.gz
          tar xzf k.tgz kustomize && sudo mv kustomize /usr/local/bin/

      - name: Bump da tag em overlays/dev
        working-directory: overlays/dev
        run: kustomize edit set image store-api=$IMAGEM:$TAG

      - name: Abre o PR
        uses: peter-evans/create-pull-request@5f6978faf089d4d20b00c7766989d076bb2fc7f1 # v8.1.1
        with:
          token: ${{ secrets.CONFIG_REPO_TOKEN }}
          branch: bump/store-api-${{ github.run_number }}
          title: "dev: store-api ${{ github.run_number }}"
          commit-message: "chore(dev): bump store-api to ${{ github.run_number }}"
          body: "Gerado pelo CI da store-api (run ${{ github.run_id }}). Testes e scan aprovados."
```

Nenhuma credencial de cluster no workflow:

```
$ grep -v '^\s*#' .github/workflows/ci.yaml | grep -ciE 'kubeconfig|kubectl'
0
```

A única ocorrência das palavras é o comentário que explica a ausência delas. O CI só tem permissão de escrita no **registry** (GITHUB_TOKEN, `packages: write`) e no **repo de config** (PAT). Quem tem credencial de cluster é o Argo CD, de dentro do cluster. As actions estão fixadas por SHA do commit, e o Trivy roda pela imagem oficial com versão fixa, não pela `trivy-action` comprometida em março de 2026.

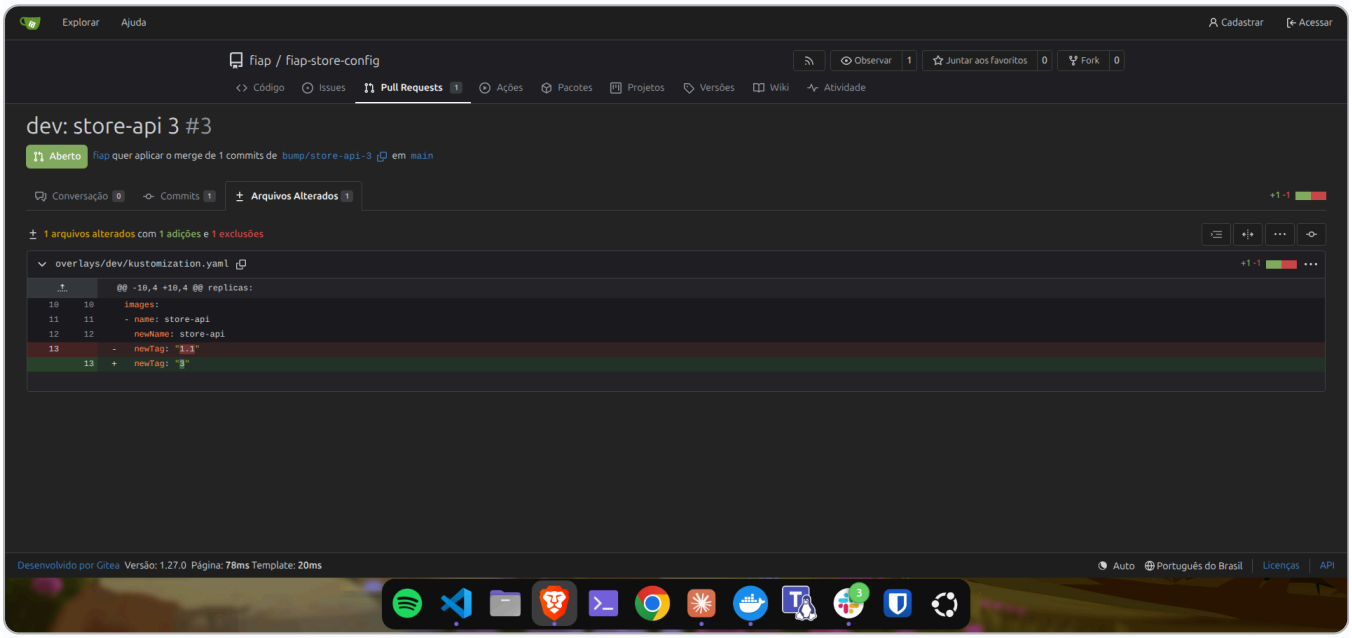
Execução local dos mesmos passos (`03-simula-ci.sh`, `RUN=3` no lugar de `github.run_number`):

```
pytest -q                     ..   2 passed
docker build ... store-api:3
trivy image --exit-code 1 --severity CRITICAL --ignore-unfixed store-api:3      → aprovado
kind load docker-image store-api:3                                              ("push")
kustomize edit set image store-api=store-api:3   (em overlays/dev)
PR #3 http://localhost:3000/fiap/fiap-store-config/pulls/3
```

Achado: na primeira execução (`RUN=2`, PR #2), o `kustomize edit` **reformatou** o arquivo inteiro (indentação, ordem das chaves) e o diff do PR ficou com 13 linhas para uma troca de tag. A correção foi pelo Git: um commit (`c712517`) deixando o overlay no

formato canônico do kustomize. A partir dele, o bump gera o diff mínimo. O PR #2 foi fechado sem merge.

```
--- a/overlays/dev/kustomization.yaml
+++ b/overlays/dev/kustomization.yaml
@@ -10,4 +10,4 @@ replicas:
 images:
 - name: store-api
   newName: store-api
-  newTag: "1.1"
+  newTag: "3"
```



Merge e sincronização sem ação no cluster:

```
21:48:05 merge do PR #3 (HTTP 200)
21:52:03 store-dev em store-api:3        (3 min 58 s, polling do Argo CD, sem refresh manual)
$ kubectl -n store-dev exec deploy/store-front -- wget -qO- http://store-api/info
{"versao":"3","env":"dev","db_host":"nao-configurado","pod":"store-api-849dc5655d-8fzlx"}
```

```
*   7c670ab Merge pull request 'dev: store-api 3' (#3) from bump/store-api-3 into main
|\
| * 61ac5a7 chore(dev): bump store-api to 3
|/
* c712517 style(dev): canonical kustomize format so ci bumps produce minimal diffs
* 37b1a82 feat: app-of-apps with infra, front, worker, presync hook and knative
* d34967d feat: add sealed db credentials for dev and prod
* 6573219 feat: add store-relatorio reading db password from secret
* 9455bbf Revert "chore(dev): bump store-api to 9.9-inexistente"
* c59dcde chore(dev): bump store-api to 9.9-inexistente
* 5fc62d4 chore: fail stuck rollouts after 90s
*   a119cc2 Merge pull request 'Promove store-api 1.1 para prod' (#1) from promove-prod-1.1 into main
```

A primeira chamada logo depois do rollout recebeu `Connection refused`. Dev tem 1 réplica e não tem `preStop`, então a troca do Pod único cai exatamente na corrida Endpoints x SIGTERM medida no 10.3. Em prod (4 réplicas e `maxUnavailable: 0`) isso não aparece.

Componente serverless

```yaml
# store-api como Knative Service: escala a zero sem trafego, ate 20 sob carga
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: store-api
spec:
  template:
    metadata:
      annotations:
        autoscaling.knative.dev/min-scale: "0"
        autoscaling.knative.dev/max-scale: "20"
    spec:
      containers:
        - image: store-api:1.1
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8000
          env:
            - name: APP_ENV
              value: serverless
          readinessProbe:
            httpGet:
              path: /ready
          securityContext:
            runAsNonRoot: true
            runAsUser: 65532
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
            seccompProfile:
              type: RuntimeDefault
```

Aplicado pelo Argo CD (Application `store-serverless`, `Synced/Healthy`) com o Knative real:

```
$ kubectl get ksvc,revision -n store-serverless
service.serving.knative.dev/store-api   http://store-api.store-serverless.svc.cluster.local   store-
api-00001   store-api-00001   True
revision.serving.knative.dev/store-api-00001   store-api   1   True   ACTUAL REPLICAS 0   DESIRED
REPLICAS 0

21:43:28 zero Pods (escalou a zero cerca de 70 s depois de ficar pronto)
$ curl http://store-api.store-serverless.svc.cluster.local/info      (de um Pod no cluster)
{"versao":"1.1","env":"serverless","db_host":"nao-configurado","pod":"store-api-00001-deployment-
657bb5d5db-klkr6"}
HTTP 200 em 8.04s   ← cold start: o activator segura a requisição enquanto o Pod sobe do zero
```

Duas adaptações para o lab: o Knative foi configurado para **não** resolver tag em digest para `docker.io` (`registries-skipping-tag-resolving`), porque a imagem só existe nos nós do kind. A validação também passou nos dois modos de dry-run: `service.serving.knative.dev/store-api created (server dry run)` e `(dry run)`.

(a) Por que `minScale: 0` não conflita com a reconciliação contínua: o Argo CD compara a **intenção** declarada (`spec` do `Service` do Knative, com as anotações `min-scale: "0"` e `max-scale: "20"`) com a `spec` viva no cluster, e as duas continuam idênticas quando o tráfego some. A contagem de Pods é **execução**: ela vive no Deployment que o Knative cria para cada Revision, que não está no Git, e o Argo não gerencia esse Deployment. O autoscaler mexe em réplicas que o Argo nunca declarou, então não há o que reconciliar. É o mesmo raciocínio do HPA no 10.4: quem escala não pode disputar o campo com quem aplica.

(b) `ignoreDifferences`: o webhook do Knative preenche na `spec` do Service campos que não estão no Git: `spec.template.spec.containerConcurrency`, `timeoutSeconds`, `enableServiceLinks` e o bloco `traffic` (`latestRevision: true`, `percent: 100`), além das anotações `serving.knative.dev/creator` e `lastModifier`. No lab, `argocd app diff store-serverless` saiu vazio (`exit=0`) sem nenhum `ignoreDifferences`. A estratégia padrão do Argo CD v3.5 é a **Legacy** (doc *Diff Strategies*): ela só compara os campos que o manifesto do Git declara, e esses defaults ficam fora da comparação. O `ignoreDifferences` passa a ser necessário em dois casos: (1) quando um campo **declarado** no Git é reescrito pelo webhook, por exemplo uma `image` por tag que o Knative troca por digest, ou o `traffic` declarado e depois alterado; e (2) quando se liga o *server-side diff* com a inclusão de mutation webhooks, porque aí os defaults aparecem no diff. Nesses casos, o certo é ignorar por `managedFieldsManagers` (o controller do Knative), que é mais preciso que listar `jsonPointers` campo a campo.

(c) Metade Knative e metade Deployment num ApplicationSet: o template do ApplicationSet gera a Application, não os manifestos. A diferença fica no **path** e nos **parâmetros**. Com gerador `git` por diretório, `overlays/serverless-*` e

`overlays/<env>` já viram Applications distintas sem mudar nada. Com gerador `list`, cada elemento ganharia um campo (`tipo: knative|deployment`) usado no `path` (`overlays/{{.tipo}}-{{.env}}`) e em `ignoreDifferences` condicional via `templatePatch`, aplicado só quando `tipo == knative`. O projeto precisaria liberar o grupo `serving.knative.dev` (já liberado, porque o `namespaceResourceWhitelist` padrão é tudo), e a health check do Knative Service é nativa do Argo.

Critérios de aceite

Critério	Resultado
Um único `kubectl apply -f` resultando em pelo menos 4 Applications `Synced/Healthy`	ok (5, em 37 s)
Namespaces com configurações comprovadamente distintas	ok (réplicas, tag, PSA, PDB, `APP_ENV`)
PR de bump criado pelo CI sem intervenção manual	**parcial**: workflow pronto e passos executados por script local, sem runner de CI no lab
Workflow sem credencial de cluster	ok (`grep` = 0)
Análise serverless: o Argo compara `spec` (intenção), não a contagem de Pods (execução)	ok